

Internet of Things (IoT) Lab

Lab 04: Benchmarking & Libraries

Georgios Mentzos, Miran Tobar, Jörg Henkel | June 18, 2026

CHAIR FOR EMBEDDED SYSTEMS (CES)

Embedded systems are resource-constrained. Code that is correct can still be too slow, too memory-hungry, or too power-hungry on a microcontroller.

In this lab you will:

- Learn **how** to benchmark and optimize code on the STM32 platform.
- Learn how and **when** to use optimized CMSIS kernels for your application.
- Learn to **judge** whether an optimization actually improved runtime.

- It is the Data Watchpoint and Trace unit of the Cortex-M core.
- It contains a 32-bit cycle counter called CYCCNT.
- More info can be found here:

<https://developer.arm.com/documentation/ka001499/1-0/>

Why DWT? To compare implementations, we need a precise, low-overhead time source. The obvious alternatives fall short:

- HAL_GetTick(): only millisecond resolution.
- Hardware timer: could work, but would cost a peripheral and additional setup.

The **DWT**'s CYCCNT register is already in the core: it counts *every* CPU cycle, with single-cycle resolution and almost no overhead.

How to Use the DWT Cycle Counter

```
static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; #Enable trace subsystem
    DWT->CYCCNT = 0; #Reset the counter
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk; # Start the counting
}
```

After enabling the counter, wrap the code you want to measure:

```
uint32_t start = DWT->CYCCNT;    #Timestamp before
/* code to benchmark */
uint32_t cycles = DWT->CYCCNT - start; #elapsed cycles calculation
```

Convert to time with the core clock: $t = \text{cycles} / f_{\text{CPU}}$ (e.g. 1600 cycles @ 160 MHz = 10 μs).

Watch out: CYCCNT is 32-bit and wraps after 2^{32} cycles (≈ 27 s at 160 MHz).

- **Algorithmic:** reduce asymptotic cost or number of operations
- **Compiler:** use suitable optimization flags
- **Data layout & Cache:** improve memory locality and alignment. On the STM32U5 the data cache covers only external (OCTOSPI) memory, while the fast internal SRAM is accessed uncached
- **Fixed-point arithmetic:** replace floating point where appropriate
- **SIMD/DSP instructions:** process multiple values per instruction
- **Libraries:** use CMSIS-DSP, CMSIS-NN, or vendor libraries
- **Memory placement:** choose SRAM, flash, external memory, or tightly coupled memory carefully

The compiler can strongly affect runtime and code size.

Typical flags:

- -O0: no optimization, useful for debugging
- -Og: debug-friendly optimization
- -O2: good default for performance
- -O3: more aggressive optimization, may increase code size
- -Os: optimize for code size
- -Ofast: aggressive floating-point assumptions, use this **carefully**

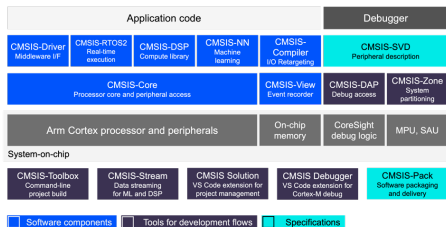
Refer to the documentation for more info on how to add compiler flags to your CMake project: https://cmake.org/cmake/help/latest/command/target_compile_options.html

`target_compile_options(app PRIVATE`

```
$(<$<CONFIG:Release>:-O3 -DNDEBUG>)
```

CMSIS (Cortex Microcontroller Software Interface Standard) is Arm's vendor-independent abstraction layer for Cortex-M cores.

- **CMSIS-Core:** core register definitions, intrinsics, startup interface
- **CMSIS-DSP:** optimized signal-processing and math kernels (filters, FFT, vector/matrix ops)
- **CMSIS-NN:** optimized neural-network kernels for Cortex-M



CMSIS components

Why use it?

- Portable across Arm Cortex-M devices
- Hand-tuned implementations for the target core (incl. SIMD/DSP instructions)

Source: https://arm-software.github.io/CMSIS_6/latest/General/index.html#cmsis_components

Integrating CMSIS: Two Options

There are two common (and a million uncommon) ways to add a CMSIS component (e.g. CMSIS-DSP) to your project.

Add the library's .c sources to your own and compile them together.

- + Built with *your* flags (`-mcpu`, `-mfpu`, `-O...`)
- + Can prune unused sources
- + Full debug visibility, no flag/ABI mismatch
- Longer build times
- You manage sources and config

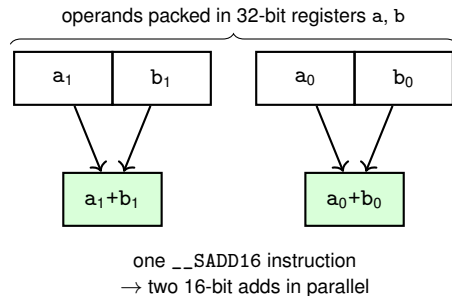
Link against a prebuilt static archive (e.g. `libCMSISDSP.a`).

- + Faster builds
- + Fewer sources to manage
- + Only build once
- Must match core, FPU, and float-ABI exactly
- Optimization flags are baked in, not yours
- Harder to debug, less flexible

CMSIS SIMD Intrinsics & Loop Unrolling

Some Cortex-M cores support DSP/SIMD instructions, exposed by CMSIS as intrinsics. E.g. `__SADD16` adds two 16-bit values packed in one 32-bit register. Combined with **loop unrolling** (several iterations' work per loop body), this cuts branch/counter overhead and the number of arithmetic operations.

```
/* scalar: 1 add / instr */  
for (i=0; i<N; i++)  
    y[i] = a[i] + b[i];  
/* SIMD: 2 adds / instr */  
for (i=0; i<N; i+=2)  
    pack(&y[i]) =  
        __SADD16(pack(&a[i]),  
                pack(&b[i]));
```



Reference:

https://arm-software.github.io/CMSIS_5/Core/html/group__intrinsic__SIMD__gr.html

The Cortex-M33 we use on the B-U585I-I0T02A has two types of cache.

Instruction cache (ICACHE)

- Speeds up instruction fetches
- Especially relevant when executing from flash or external memory
- 8 KB, on the Cortex-M33 C-AHB bus, covers code/data fetched from internal flash and SRAM (fast bus) and external memory (slow bus)

Data cache (DCACHE1)

- Speeds up repeated data accesses to/ from external memories
- Requires care when DMA or peripherals access the same memory
- On the STM32U5: 4 KB, on the S-AHB bus. **Caches *only* external memory (OCTOSPI/FMC, region 0x60000000–0x9FFFFFFF).** Internal SRAM is accessed uncached.

Further detail: ST application note **AN5212** (*How to use the STM32 cache to optimize performance and power efficiency*). As well as the reference manual.

HAL and many STM32 libraries are C libraries. However sometimes we want to use a C++ library with our projects in these cases we need to, tell the C++ compiler to use C linkage.

C and C++ can be linked together, but they do not use the same symbol names by default. C++ supports function overloading while C has simpler symbol names. Therefore, the compiler changes function names internally. This is called **name mangling**.

```
#ifdef __cplusplus
extern "C" {

void uart_print(const char* s);

#ifdef __cplusplus
}
#endif
```

C and C++ in CMake

If your project contains C++ files, enable CXX in CMake:

```
project(iot_lab C CXX ASM)
```

Add C and C++ sources normally:

```
target_sources(app PRIVATE
    main.cpp
    uart.c
    sensor.c
)
```

Set the C++ standard if needed:

```
target_compile_features(app PRIVATE cxx_std_17)
```

Tasks for Today

1. Manually benchmark and optimize two kernels.
2. Fast Fourier transform for real time audio-signals.

The goal of these tasks is for you to get some experience with benchmarking your projects, get to know some of the easier optimization methods and learn when to fall back on optimized libraries such as CMSIS.

Task 01: Setup

In this task, you will optimize two different kernels using some of the basic optimization techniques discussed in this lab and benchmark the results:

1. Set up the board at 160 MHz, with the instruction cache (ICACHE) enabled in CubeMx.
2. Download `iot_kernel.h`, `iot_kernel.c`, `iot_test_data.h` from Ilias.
3. Copy the headers into `Inc/`, the source into `Src/`.
4. Build and confirm it compiles cleanly.

```
cmake --preset Debug
cmake --build --preset Debug
```

`iot_test_data.h` holds fixed inputs *and* golden outputs (`Y_AXPY_GOLD`, `Y_CONV_GOLD`, plus `N_AXPY`, `XLEN_CONV`, `KLEN_CONV`) for deterministic correctness checks.

Task 01: DWT & Benchmark baseline

Use the DWT cycle counter to time the baseline `axpy_u8` and `conv1d_i16_valid` on the data from `iot_test_data.h`; wrap the timing in a small helper, bracketing the code under test with barriers (see the memory-ordering slide). A couple rules to follow:

- **Warm up first**, then measure.
- **Run each benchmark ≥ 5 times**, then report the **minimum (or median)**.
- **Defeat dead-code elimination.** At $\geq -O2$ the compiler may delete a call whose result is unused, consume the output (fold `y` into a printed checksum, or compare to the golden array).
- **Verify correctness** against the golden arrays.

Report cycle counts, the **-O level**, and the **repetition count** for later comparison.

DWT registers (Cortex-M33 TRM): <https://developer.arm.com/documentation/100230/0004/>

Task 01: Loop Unrolling for axpy_u8

axpy (“a times x plus y”) is the scalar version of saxpy:

$$y_i \leftarrow (a \cdot x_i + y_i) \bmod 256, \quad i = 0, \dots, n - 1$$

Your task is to implement a loop-unrolled version of AXPY; any factor ≥ 4 is fine. Note the compiler can sometimes unroll on its own at -O3, so below that any unrolling must be done in source, you do not need to use any intrinsics for this task:

```
void axpy_u8_unrolled(size_t n, uint8_t a,  
                      const uint8_t *x, uint8_t *y);
```

- Keep the semantics identical to the baseline (modular `uint8_t` arithmetic).
- Handle the **remainder** correctly when `n` is not a multiple of your unrolling factor.

Arm's opinion on loop unrolling: <https://developer.arm.com/documentation/dui0773/1/Coding-Considerations/Loop-unrolling-in-C-code>

Task 01: SIMD for conv1d_i16_valid

conv1d is a *valid* 1D convolution - a sliding dot product of the signal x against a length- k filter h , the core operation of FIR filters:

$$y_t = \sum_{i=0}^{k-1} x_{t+i} h_i, \quad t = 0, \dots, x_{\text{len}} - k$$

Your task is to implement a SIMD version using Arm DSP intrinsics. Process **two 16-bit values at once** with `__SMLAD`, read the documentation for more info:

```
void conv1d_i16_valid_opt(const int16_t *x, size_t x_len,
                          const int16_t *h, size_t k_len,
                          int32_t *y);
```

For more info on the available intrinsics checkout:

<https://developer.arm.com/documentation/100235/0100/The-Cortex-M33-Instruction-Set/General-data-processing-instructions/List-of-data-processing-instructions>, as well as:
https://arm-software.github.io/CMSIS_5/Core/html/group__intrinsic__SIMD__gr.html

Task 01: conv1d_i16_valid continued

Alignment: `int16_t x[]` is only 2-byte aligned, so `&x[t]` is 4-byte aligned only for *even* `t`. A raw `(uint32_t*)` cast at an odd `t` is **undefined behaviour**. The compiler may assume alignment and emit a multi-word load (LDM/LDRD) that **faults** on the M33. Even when it works, an unaligned access splits into extra bus transfers and skews timing.

- Output must match the baseline (compare to `Y_CONV_GOLD`).
- **Safe load**, pick one:
 - `memcpy(&w, &x[t], 4)` is canonical and lowers to one load.
 - `__UNALIGNED_UINT32_READ(&x[t])` is the CMSIS shorthand.
- **Tail:** `__SMLAD` consumes two values at a time, so an odd `k_len` leaves the last value with no partner, so don't forget to add a **remainder loop**.

Unaligned accesses (M33 TRM): <https://developer.arm.com/documentation/100230/0100/Programmers-Model/Memory-model/Unaligned-accesses>

Task 01: Benchmark Optimized Kernels

Lastly, re-run the benchmark harness from the beginning on your optimized kernels `axpy_u8_unrolled` and `conv1d_i16_valid_opt`, keeping the **same conditions** (same inputs, same `-O` level, same repetition count) so the numbers are comparable.

You should:

1. Measure each optimized kernel and compute the **speedup** relative to its Task 1.2 baseline.
2. For any kernel that is *not* faster, explain the cause.

Task 01 Bonus: Byte SIMD (SAD)

The *sum of absolute differences* is the core operation of image block-matching and template matching. The baseline reduces two byte arrays to a single scalar:

```
uint32_t sad_u8(size_t n,                /* baseline */
               const uint8_t *a, const uint8_t *b);
uint32_t sad_u8_opt(size_t n,            /* your version */
                   const uint8_t *a, const uint8_t *b);
```

Your task: implement `sad_u8_opt` with `__USADA8`, which computes $|a_0 - b_0| + \dots + |a_3 - b_3|$ over four byte lanes and adds the running accumulator, all in one instruction. Use alignment-safe 32-bit loads with a scalar tail, and verify against `SAD_GOLD`.

Question: you should see much more speedup compared to the ≈ 2 from the `int16` convolution. Explain why byte SIMD pays off so much more here. (Hint: which packed operations does the Cortex-M33 provide for bytes, and which does it *not*?)

Task 02: Background

The **Discrete Fourier Transform (DFT)** maps a sampled signal from the time domain to the frequency domain. It is used in audio analysis (spectrograms, pitch detection, equalisers), vibration and condition monitoring, communications (OFDM), and fast convolution in the frequency domain. Evaluated directly, bin k is

$$X_k = \sum_{n=0}^{N-1} x_n \left(\cos \frac{2\pi kn}{N} - j \sin \frac{2\pi kn}{N} \right), \quad k = 0, \dots, N-1,$$

Your task: is to implement this direct DFT as a baseline and benchmark it on a real data load. When it proves too slow (spoiler: it will), replace it with the optimized **Fast Fourier Transform (FFT)** from the CMSIS-DSP library, which computes the same spectrum in $\mathcal{O}(N \log N)$ by exploiting the symmetry and periodicity of the DFT (Cooley–Tukey).

Task 02: Setup

1. Configure the board for **160 MHz** with the instruction cache (ICACHE) enabled in CubeMX.
2. Configure UART DMA on **DMA channels 2 and 3** in CubeMX and enable the corresponding NVIC interrupts. Enable both the **USART global interrupt** and the DMA channel interrupt. See the previous lab for the UART DMA configuration as a reference.
3. Add the BSP files to both the top-level project and the CMake build, and include the audio header.
4. Add CMSIS-DSP as a git submodule under `external/cmsis-dsp`:
`https://github.com/ARM-software/CMSIS-DSP`. See the week-1 material for the expected project structure.

Task 02: Setup Audio Capturing

As you learned in last week's lab, the board carries a **PDM digital microphone**. The ADF peripheral decimates the 1-bit PDM bitstream into 16-bit PCM samples at 16 kHz. The BSP fills a buffer continuously using circular DMA, with half-transfer and full-transfer callbacks notifying the CPU each time half the buffer fills. Declare a buffer sized to hold 1024 samples ($2 * \text{FFT_SIZE}$). Since `FFT_SIZE` is 512, each callback fires once exactly 512 new samples are ready.

```
void BSP_AUDIO_IN_HalfTransfer_CallBack(uint32_t Inst);      /* lower half */  
void BSP_AUDIO_IN_TransferComplete_CallBack(uint32_t Inst); /* upper half */
```

- Use the callbacks to set a flag for the main loop. **No FFT, no printing, no UART** inside an interrupt.
- If a new block arrives before the previous one was processed, increment an global overrun counter. This is your signal that the CPU cannot keep up.

Task 02: Naive DFT Baseline

Next implement the naive DFT. Use `sinf` and `cosf` from `math.h`.

```
void dft_naive(const float32_t *in, float32_t *out, uint32_t n);
```

1. Convert the `int16_t` samples to `float32_t` in a separate loop.
2. Input `FFT_SIZE` samples and output `FFT_SIZE / 2` magnitudes.
3. Optional: find the **peak bin** (start at bin 1 to skip DC) and convert to Hz.

Hint: handling the complex exponential

You do not need `complex.h`. By Euler's formula $e^{-i\theta} = \cos \theta - i \sin \theta$, each bin $X_k = \sum_n x_n e^{-i2\pi kn/N}$ splits into two real sums, accumulated in separate `float32_t` variables. Here k is the bin index and N is the total number of samples:

$$\operatorname{Re}(X_k) = \sum_n x_n \cos \theta_{k,n} \quad \operatorname{Im}(X_k) = - \sum_n x_n \sin \theta_{k,n}$$

with $\theta_{k,n} = 2\pi kn/N$. The magnitude is then $|X_k| = \sqrt{\operatorname{Re}(X_k)^2 + \operatorname{Im}(X_k)^2}$.

Task 02: Measuring Performance

Now measure the performance of your naive implementation. How many cycles does it take to complete the FFT? Use the DWT cycle counter with the same helper from Task 01, then convert cycles into time and compare against the real-time budget.

```
uint32_t t0 = DWT->CYCCNT;  
dft_naive(in, mag, FFT_SIZE);  
uint32_t cycles = DWT->CYCCNT - t0;
```

The CPU must finish one block before the DMA refills it:

$$T_{\text{block}} = \frac{N}{F_s} = \frac{512}{16000} = 32 \text{ ms} \quad C_{\text{budget}} = T_{\text{block}} \cdot f_{\text{CPU}} = 5\,120\,000 \text{ cycles}$$

$$\text{load} = \frac{\text{cycles}}{C_{\text{budget}}} \times 100 \%$$

Report cycles, time in ms, and load percent.

Task 02: Detecting Overruns

If load exceeds 100%, the naive DFT cannot keep up and the CPU starts falling behind DMA. Confirm this on the board by counting overruns. Add a `volatile uint8_t fft_done` flag, initialized to 1. Each callback checks this flag right when a new half becomes ready. If `fft_done` is still 0, the fft has not finished processing the previous half before DMA refilled it, so increment `overrun`. The callback then clears the flag, and the main loop sets it back to 1 once `dft_naive` returns.

```
volatile uint32_t overrun = 0;
volatile uint8_t  fft_done = 1; /* idle at start */

void HAL_ADF_AcqHalfCpltCallback(...) {
    if (!fft_done) overrun++;
    fft_done = 0;
    new_half = 0; /* first half ready */
}
...
```

Task 02: Streaming Results over UART DMA

Send one compact report per processed block over **UART using DMA**, so the CPU is free while bytes go out.

```
HAL_UART_Transmit_DMA(&huart1, buf, len);
```

- Start a new transfer only when the previous one finished. Check that `HAL_UART_GetState` equals `HAL_UART_STATE_READY`.
- If a report is ready while the line is still busy, **drop it** and count a drop. Do not block the FFT.
- As a small extra task you could also report the number of dropped messages via UART, feel free to get creative with the format.

Task 02: Adding CMSIS-DSP

Now you will bring in the optimized FFT from CMSIS-DSP, which computes the same spectrum in $\mathcal{O}(N \log N)$.

```
set(CMSISDSP_ROOT ${CMAKE_CURRENT_SOURCE_DIR}/../../external/CMSIS-DSP)

target_sources(${CMAKE_PROJECT_NAME} PRIVATE
    ${CMSISDSP_ROOT}/Source/TransformFunctions/TransformFunctions.c
    ${CMSISDSP_ROOT}/Source/ComplexMathFunctions/ComplexMathFunctions.c
    ${CMSISDSP_ROOT}/Source/CommonTables/CommonTables.c
)

target_include_directories(${CMAKE_PROJECT_NAME} PRIVATE
    ${CMSISDSP_ROOT}/Include/
    ${CMSISDSP_ROOT}/PrivateInclude/
)

target_compile_definitions(${CMAKE_PROJECT_NAME} PRIVATE
    ARM_MATH_CM33
)
```

Task 02: Initializing the FFT

Benchmark builds need **-O3** and **-ffast-math** so the naive-vs-CMSIS cycle comparison reflects an optimized FFT. Keep Debug at **-O0 -g3** so the code stays steppable.

```
target_compile_options(${CMAKE_PROJECT_NAME} PRIVATE
    $<$<CONFIG:Release>:-O3>
    $<$<CONFIG:Release>:-ffast-math>
)
```

Then initialize the FFT instance once at startup and reuse it every block:

```
#include "arm_math.h"
static arm_rfft_fast_instance_f32 rfft;
arm_rfft_fast_init_f32(&rfft, FFT_SIZE);    /* once at startup */
```

Task 02: Using the CMSIS FFT

The transform and the magnitude are **two separate steps**. The real FFT returns a **complex** spectrum, then `arm_cmplx_mag_f32` reduces each bin to its magnitude.

```
arm_rfft_fast_f32(&rfft, fft_input, fft_output, 0);      /* time -> complex */
fft_magnitude[0] = fabsf(fft_output[0]);                /* DC, real only */
arm_cmplx_mag_f32(&fft_output[2], &fft_magnitude[1], FFT_BINS - 1);
```

Output packing of `arm_rfft_fast_f32` (real input):

- `fft_output[0]` is bin 0 (DC), real only.
- `fft_output[1]` is the Nyquist bin, real only, packed into the freed slot. For this demo you ignore it.
- From index 2 onward the bins are interleaved (real, imag) pairs.

Keep the **same buffers and the same DC removal** as the naive path, so the two modes are directly comparable. You can read more here: https://arm-software.github.io/CMSIS_5/DSP/html/group__RealFFT.html

Task 02: Benchmark the FFT & Bonus

Now run the same benchmark on the optimized version. Make sure you select the Release CMake configuration. **You should find** the naive DFT far over the 32 ms budget with overrun climbing, while the CMSIS FFT handles the sample rate well. As a bonus you can turn the report line into a live **spectrum analyser**. Collapse the FFT_BINS magnitudes into a small number of bands, take the loudest bin in each band, and map it to a height with a log scale.

```
const char *glyph[9] = { " ", "_", ".", ..., "#" }; /* low to high */  
int lvl = (m > 1.0f) ? (int)(log2f(m) * 0.9f) : 0; /* clamp 0..8 */
```

Ideas:

- Print one row of band glyphs per block. Whistle a slow glide and watch the loud band sweep across the row.
- Build the band string only for reports you actually send, so the work stays off the path of dropped reports.

UTF-8 block elements (U+2581 . . U+2588) render as bars in a UTF-8 terminal.

Questions?

Chair for Embedded Systems (CES)
Karlsruhe Institute of Technology (KIT)